

# Notification App – Three Tier Architecture

## Problem Statement:

Create a console application for a company that wants to send notifications to users. The company currently supports:

- \* Email notification

- \* SMS notification

In the future, they may add WhatsApp notification.

## Requirements

### 1. Presentation Layer

Create a console menu that allows the user to:

1. Add user details
2. Choose notification type
  - \* Email
  - \* SMS
3. Enter notification message
4. Send notification
5. Display sent notification details

---

### 2. Business Logic Layer

Create business rules before sending notification:

1. Message should not be empty.
2. Message length should be at least 5 characters.
3. Email notification should be sent only if the user has a valid email.
4. SMS notification should be sent only if the user has a valid phone number.
5. SMS message should not exceed 160 characters.
6. A sent date should be added automatically.
7. The system should decide which notification class to use based on the selected notification type.

---

### 3. Data Access Layer

Since this is a console application, no database is required.

Create a data access class that stores sent notifications in a collection such as:

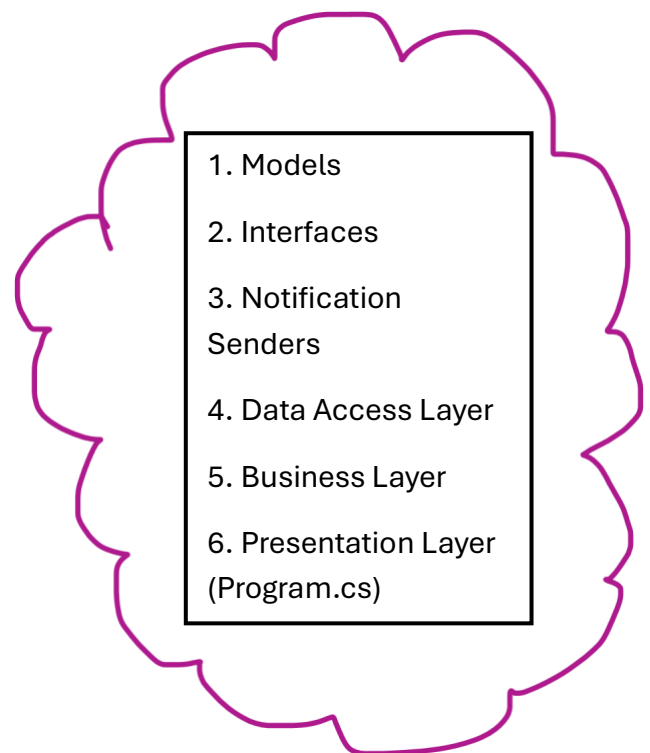
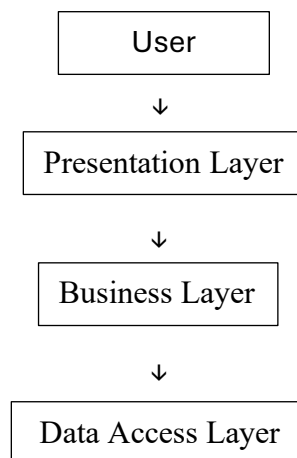
```
```csharp
List<Notification>
```
```

The data layer should support:

1. Save notification
2. Get all sent notifications

---

## Three-Tier Architecture Design:



### 1. Presentation Layer:

- Program.cs – what the user will see, the menu , reading inputs and displaying outputs

### 2. Business Layer:

- NotificationService.cs – business logic written here
  - SendNotification()
  - ValidateUser()
  - ValidateMessage()
  - Rules()

Name – Sai Nikitha NSR

Date – 7<sup>th</sup> May 7, 2026

Day 10 Three Tier Architecture – Notification App

Mail – sainikinsr@gmail.com

- ChooseSender()
- NotificationExceptions.cs

### 3. Data Access Layer

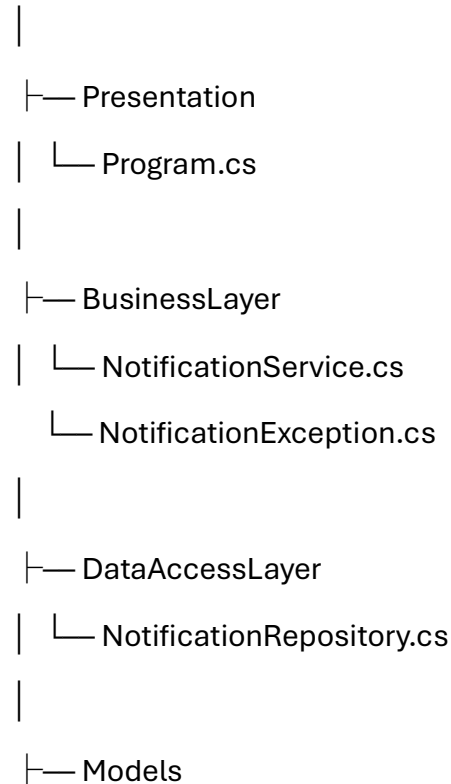
- NotificationRepository.cs
  - IRepository interface -> CRUD opns

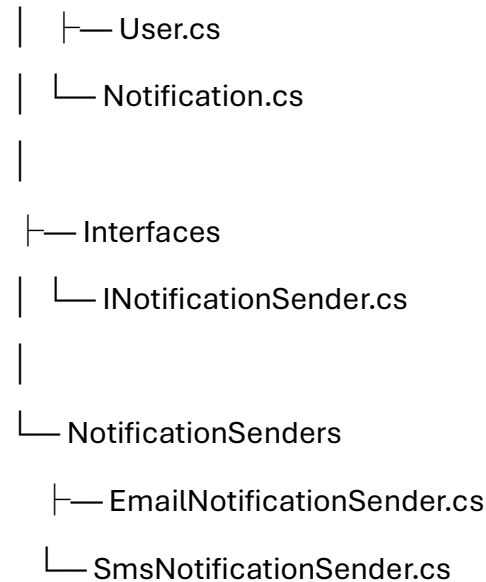
## Entity Classes

1. User
  - Person receiving notification
  - Prop : Name, mail, ph no.
2. Notification
  - Content of message
  - Prop : Message, SentDate, Type

## Folder Structure

SimpleNotificationSystem





## 1. Models

- **Notification.cs**
- Objects: Message, SentDate,
- Methods: GetAuditLine(), HasContent()
- **User.cs**
- Objects: Name, Email, Phone
- Methods: HasValidMail(), HasValidPhone(), GetShortInfo()

## 2. Interfaces

- **INotificationSender**
- Methods: void Send(User user, Notification notification);
- Later runtime polymorphism implemented by EmailNotificationSender and SMSNotificationSender.
- Pseudocode
  - 1. Common notification sending interface
  - 2. Any notification sender must implement Send()
  - 3. Send method receives:
    - a. User details
    - b. Notification details
  - 4. Different sender classes will provide their own implementation
- **IRepository.cs**
- Methods: Add(), GetAll(), GetByIndex(), Update(), Delete()
- Pseudocode
  - 1. Define notification storage operation

2. CRUD opns methods.
3. Actual storage implementation will be handled by repository class.

### 3. NotificationSenders – Concrete classes

- **EmailNotificationSender.cs** and **SMSNotificationServer.cs**
- Send() method.

### 4. BAL (Business Access Layer)

#### NotificationExceptions.cs

- NotificationValidationException
- NotificationProcessException

#### Purpose

- Used for custom exception handling in the business layer.
- Separates validation errors from processing/system errors.

#### Concepts Used

- Inheritance
- Custom Exceptions
- Constructor Chaining
- Exception Handling

#### Pseudocode

##### *NotificationValidationException*

1. Create custom validation exception class
2. Inherit from base Exception class
3. Accept custom error message
4. Pass message to parent Exception constructor

##### *NotificationProcessException*

1. Create custom processing exception class
2. Inherit from base Exception class
3. Accept:
  - a. Error message
  - b. Original exception

4. Pass both values to parent Exception constructor
5. Used for unexpected processing failures

## 5. DAL (Data Access Layer)

### NotificationRepository.cs

#### Methods

- Add()
- GetAll()
- GetByIndex()
- Update()
- Delete()

#### Purpose

- Stores notifications in memory.
- Handles CRUD operations.
- Acts as repository layer between BAL and storage.

#### Concepts Used

- Repository Pattern
- Generic Collections (`List<T>`)
- CRUD Operations
- Encapsulation
- Index Validation

#### Pseudocode

##### *Add()*

1. Receive notification object
2. Check if notification is null
3. Add notification into collection

##### *GetAll()*

1. Retrieve all stored notifications
2. Return copied notification list

### *GetByIndex()*

1. Receive notification index
2. Check whether index exists
3. Return matching notification
4. Return null if invalid index

### *Update()*

1. Receive index and updated notification
2. Validate index range
3. Replace old notification with updated notification

### *Delete()*

1. Receive notification index
2. Validate index range
3. Remove notification from collection

## 6. Presentation Layer

### Program.cs

#### Purpose

- Handles user interaction through console.
- Collects input from user.
- Displays menu and output messages.
- Calls business layer methods.

#### Concepts Used

- Looping
- Switch Case
- Object Creation
- Exception Handling
- Console Input/Output
- Layer Communication

#### Pseudocode

1. Start application
2. Create repository object

Name – Sai Nikitha NSR

Date – 7<sup>th</sup> May 7, 2026

Day 10 Three Tier Architecture – Notification App

Mail – sainikinsr@gmail.com

3. Create notification service object
4. Display menu repeatedly using loop

## Menu Operations

### *Send Notification*

1. Get user details
2. Get notification message
3. Get notification type
4. Call business layer send method
5. Display success/error message

### *View All Notifications*

1. Fetch all notifications from repository
2. Display notification list

### *View Notification By Index*

1. Get notification index
2. Retrieve matching notification
3. Display notification details

### *Update Notification*

1. Get notification index
2. Get updated message
3. Call update method
4. Display update status

### *Delete Notifications*

1. Get notification index
2. Delete selected notification
3. Display delete status

### *Exit*

1. Stop loop
2. Close application

*Github Link : [Click here](#)*

Name – Sai Nikitha NSR  
Date – 7<sup>th</sup> May 7, 2026

Day 10 Three Tier Architecture – Notification App  
Mail – sainikinsr@gmail.com

Name – Sai Nikitha NSR  
Date – 7<sup>th</sup> May 7, 2026

Day 10 Three Tier Architecture – Notification App  
Mail – sainikinsr@gmail.com